

CSS Avanzado y Frameworks de Estilo

Introducción

En esta sesión vas a trabajar con técnicas modernas de CSS que se usan en proyectos reales. El hilo conductor es una página de perfil llamada **DevCard** que irás construyendo y mejorando parte por parte.

⚠ LO QUE NECESITAS

Un editor de código (VS Code recomendado), Google Chrome con DevTools y conexión a internet para la Parte 4.

Objetivos de Aprendizaje

La guía tiene cuatro partes que se conectan entre sí:

1. **Parte 1** — Debugging con DevTools: encontrar y corregir errores en CSS
2. **Parte 2** — Variables CSS: crear un sistema de diseño reutilizable + modo oscuro
3. **Parte 3** — Responsive Design: que DevCard funcione en cualquier pantalla
4. **Parte 4** — Frameworks: reconstruir un componente con Bootstrap y con Tailwind

Setup del Proyecto

Descarga los archivos iniciales

📦 [Descargar archivos de inicio \(devcard-starter.zip\)](#)

O si prefieres, crea manualmente la siguiente estructura de carpetas y archivos:

```
devcard/  
├── index.html  
└── style.css
```

Archivo Base — index.html

Copia esta estructura en tu `index.html`. Es una página de perfil sencilla con navbar, hero, tarjetas de habilidades y footer.

```
<!-- DevCard – página de perfil de desarrollador -->  
<!DOCTYPE html>  
<html lang="es">  
<head>  
  <meta charset="UTF-8">  
  <meta name="viewport" content="width=device-width, initial-scale=1.0">  
  <title>DevCard</title>  
  <link rel="stylesheet" href="style.css">  
</head>  
<body>  
  
  <!-- Navbar -->  
  <nav class="navbar">  
    <span class="navbar__logo">DevCard</span>  
    <ul class="navbar__links">  
      <li><a href="#">Inicio</a></li>  
      <li><a href="#skills">Skills</a></li>  
      <li><a href="#contact">Contacto</a></li>  
    </ul>
```

```
</nav>

<!-- Hero -->
<section class="hero">
  
  <h1 class="hero__name">Ada Lovelace</h1>
  <p class="hero__role">Full-Stack Developer</p>
  <a href="#" class="btn">Ver proyectos</a>
</section>

<!-- Skills -->
<section class="skills" id="skills">
  <h2>Mis habilidades</h2>
  <div class="skills__grid">
    <div class="skill-card">
      <span class="skill-card__icon">🌐</span>
      <h3>HTML & CSS</h3>
      <p>Semántica, Flexbox, Grid y diseño responsivo.</p>
    </div>
    <div class="skill-card">
      <span class="skill-card__icon">⚙️</span>
      <h3>JavaScript</h3>
      <p>ES6+, DOM, Fetch API y programación asíncrona.</p>
    </div>
    <div class="skill-card">
      <span class="skill-card__icon">🗄️</span>
      <h3>Node.js</h3>
      <p>APIs REST, Express y manejo de bases de datos.</p>
    </div>
  </div>
</section>

<!-- Footer -->
<footer id="contact">
  <p>✉️ ada@devcard.io – GitHub: @ada</p>
</footer>

</body>
</html>
```

CSS Inicial — style.css

Este CSS tiene estilos básicos para que la página se vea funcional. En la Parte 2 lo vamos a refactorizar usando variables.

```
/* === DevCard – style.css === */

*, *::before, *::after {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}

body {
  font-family: system-ui, sans-serif;
  background-color: #f0f2f5;
  color: #1c1e21;
}

/* Navbar */
.navbar {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 1rem 2rem;
  background-color: #1a73e8;
}
```

```
.navbar__logo {
  color: #ffffff;
  font-weight: 800;
  font-size: 1.25rem;
}

.navbar__links {
  list-style: none;
  display: flex;
  gap: 1.5rem;
}

.navbar__links a {
  color: rgba(255,255,255,0.85);
  text-decoration: none;
  font-size: 0.9rem;
}

/* Hero */
.hero {
  text-align: center;
  padding: 4rem 1rem;
  background-color: #ffffff;
}

.hero__avatar {
  width: 120px;
  height: 120px;
  border-radius: 50%;
  margin: 0 auto 1rem;
  border: 4px solid #1a73e8;
}

.hero__name {
  font-size: 2rem;
  font-weight: 800;
  margin-bottom: 0.5rem;
}

.hero__role {
  color: #5f6368;
  margin-bottom: 1.5rem;
}

.btn {
  display: inline-block;
  background-color: #1a73e8;
  color: #ffffff;
  padding: 0.75rem 2rem;
  border-radius: 8px;
  font-weight: 700;
  text-decoration: none;
}

/* Skills */
.skills {
  padding: 3rem 2rem;
  max-width: 900px;
  margin: 0 auto;
}

.skills h2 {
  font-size: 1.5rem;
  margin-bottom: 1.5rem;
}

.skills__grid {
  display: flex;
  gap: 1.5rem;
```

```
}

.skill-card {
  flex: 1;
  background-color: #ffffff;
  border-radius: 12px;
  padding: 1.5rem;
  border: 1px solid #dadde1;
}

.skill-card__icon {
  font-size: 2rem;
  display: block;
  margin-bottom: 0.75rem;
}

.skill-card h3 {
  font-size: 1rem;
  margin-bottom: 0.4rem;
}

.skill-card p {
  font-size: 0.875rem;
  color: #5f6368;
}

/* Footer */
footer {
  text-align: center;
  padding: 2rem;
  color: #5f6368;
  font-size: 0.875rem;
  border-top: 1px solid #dadde1;
}
```

💡 VERIFICA ANTES DE CONTINUAR

Abre `index.html` en Chrome. Deberías ver el navbar azul, el avatar, el nombre "Ada Lovelace" y las tres tarjetas de habilidades.

Parte 1: Debugging con DevTools

Las DevTools de Chrome son la herramienta más importante para un desarrollador frontend. En esta parte vas a aprender a usarlas para encontrar y corregir errores CSS sin tocar el editor.

Abre las DevTools

Con tu página abierta en Chrome, presiona **F12** (o clic derecho → *Inspeccionar*). Asegúrate de estar en la pestaña **Elements**.

❗ LAS TRES ÁREAS PRINCIPALES DE LA PESTAÑA ELEMENTS

- **Panel izquierdo:** el árbol HTML de la página.
- **Panel derecho** → **Styles:** el CSS aplicado al elemento seleccionado.
- **Panel derecho** → **Computed:** el valor final calculado de cada propiedad.

Ejercicios Prácticos

1. Inspecciona el navbar

Haz clic sobre el logo "DevCard" en la página. En el panel **Styles** verás los estilos de `.navbar__logo`. Prueba cambiar el valor de `font-size` directamente ahí — haz doble clic sobre el número y escribe uno nuevo. Los cambios son **temporales** y se pierden al recargar.

2. Encuentra la propiedad que no aplica

Agrega temporalmente esta regla a tu `style.css`, guarda y recarga:

```
.hero__name {
  font-size: 2rem;
  font-weight: 800;
  colour: #1a73e8; /* ← error intencional */
}
```

Inspecciona el `.hero__name` en DevTools. Verás que `colour` aparece con un **ícono de advertencia** ⚠️ — CSS no reconoce esa propiedad porque la escritura correcta es `color` (sin "u"). Corrígela en tu archivo.

3. Detecta estilos sobrescritos (tachados)

Agrega esto a tu CSS, guarda y recarga:

```
.btn {
  background-color: red; /* ← regla nueva más abajo */
}
```

Inspecciona el botón. En el panel Styles verás el `background-color: #1a73e8` **tachado** — significa que fue sobrescrito. La regla que aparece más abajo (o es más específica) siempre gana. Borra la regla que agregaste.

4. Usa el emulador de dispositivos

En la barra de DevTools, haz clic en el ícono de **dispositivo móvil** (o presiona `Ctrl+Shift+M`). Selecciona "iPhone 12" en el menú desplegable. Notarás que las tres tarjetas de habilidades se salen de la pantalla — las arreglaremos en la Parte 3.

 **CHECKPOINT — PARTE 1**

 **Antes de continuar, verifica que:**

- Sabes abrir DevTools y navegar a la pestaña Elements
- Puedes editar CSS en vivo desde el panel Styles
- Identificas propiedades con error (⚠️) y estilos sobrescritos (tachados)
- Sabes activar el emulador de dispositivos móviles

Parte 2: Variables CSS

En tu `style.css` actual, el color `#1a73e8` aparece **cinco veces**. Si el cliente pide cambiar el azul por verde, tienes que editar cinco líneas. Las variables CSS resuelven esto.

¿Qué es una variable CSS?

Una variable CSS (también llamada *custom property*) almacena un valor que puedes reutilizar en todo el archivo. Se declaran con doble guión: `--nombre-variable` y se usan con `var()`.

```
/* Declarar la variable */
:root {
  --color-primary: #1a73e8;
}

/* Usar la variable */
.btn {
  background-color: var(--color-primary);
}
```

⚠️ ¿POR QUÉ `:root`?

`:root` es el selector del elemento más alto del documento (el `<html>`). Al declarar variables ahí, están disponibles en **todo** el archivo CSS. Si las declaras dentro de `.btn`, solo funcionan dentro de ese selector.

Refactoriza tu style.css

Sustituye la parte superior de tu `style.css` con este bloque de variables y luego reemplaza todos los valores hardcodeados por sus variables:

```
/* === Sistema de diseño – variables globales === */
:root {
  /* Colores */
  --color-primary: #1a73e8;
  --color-bg: #f0f2f5;
  --color-surface: #ffffff;
  --color-text: #1c1e21;
  --color-muted: #5f6368;
  --color-border: #dadde1;

  /* Tipografía */
  --font-base: system-ui, sans-serif;

  /* Espaciado */
  --space-sm: 0.5rem;
  --space-md: 1rem;
  --space-lg: 1.5rem;
  --space-xl: 2rem;
  --space-2xl: 3rem;

  /* Bordes */
  --radius-sm: 8px;
  --radius-lg: 12px;
}
```

Ahora reemplaza los valores hardcodeados. Por ejemplo:

```
/* Antes */
body {
  background-color: #f0f2f5;
  color: #1c1e21;
}

/* Después */
body {
  background-color: var(--color-bg);
  color: var(--color-text);
}
```

Haz lo mismo con **todos** los selectores: `.navbar`, `.hero__avatar`, `.btn`, `.skill-card`, etc.

⚠️ PRUEBA QUE FUNCIONE

Cambia temporalmente el valor de `--color-primary` de `#1a73e8` a `#e8341a`. Si el navbar, el borde del avatar y el botón cambian a rojo al mismo tiempo con **una sola edición**, tu refactorización es correcta. Regresa el valor a azul.

Modo Oscuro con Variables

La ventaja más poderosa de las variables es que puedes sobrescribirlas para un selector específico. Agrega un botón de modo oscuro y este CSS:

En `index.html` — agrega en el navbar:

```
<button id="toggle-dark" class="btn btn--sm">🌙 Modo oscuro</button>
```

En style.css — agrega al final:

```
/* === Modo oscuro ===
Al agregar la clase .dark al body,
las variables se sobreescriben.
El resto del CSS no cambia. */

body.dark {
  --color-bg: #18191a;
  --color-surface: #242526;
  --color-text: #e4e6eb;
  --color-muted: #b0b3b8;
  --color-border: #3a3b3c;
}

.btn--sm {
  padding: 0.4rem 1rem;
  font-size: 0.8rem;
}
```

En index.html — agrega antes de </body>:

```
<script>
const btn = document.getElementById('toggle-dark');
btn.addEventListener('click', () => {
  document.body.classList.toggle('dark');
  btn.textContent = document.body.classList.contains('dark')
    ? '☀️ Modo claro' : '🌙 Modo oscuro';
});
</script>
```

💡 CHECKPOINT — PARTE 2

✅ Antes de continuar, verifica que:

- Tu `style.css` tiene todas las variables declaradas en `:root`
- No hay valores de color ni espaciado hardcoded fuera de `:root`
- Al cambiar `--color-primary` en un lugar, todo cambia
- El botón de modo oscuro funciona sin modificar ninguna regla de CSS adicional

Parte 3: Responsive Design 📱

Si abres DevCard en el emulador de iPhone (lo hiciste en la Parte 1), las tarjetas se salen de la pantalla. Vamos a arreglarlo con un enfoque **mobile-first**: diseñamos primero para pantalla pequeña y expandimos con media queries.

¿Qué es mobile-first?

Escribes los estilos *base* para móvil (sin media query) y luego agregas reglas adicionales para pantallas más grandes usando `min-width`. Al revés (desktop-first) usarías `max-width`.

```
/* Mobile-first: el estilo base es para móvil */
.skills__grid {
  display: flex;
  flex-direction: column; /* columna en móvil */
  gap: var(--space-lg);
}
```

```
/* A partir de 768px: fila (tablet y desktop) */
@media (min-width: 768px) {
  .skills__grid {
    flex-direction: row;
  }
}
```

Media Queries para DevCard

Modifica tu `style.css` para que las reglas base sean mobile-first y agrega los dos breakpoints siguientes:

```
/* === Mobile-first: estilos base para pantallas pequeñas === */

.navbar {
  flex-direction: column;
  gap: var(--space-md);
  padding: var(--space-md);
  text-align: center;
}

.navbar__links {
  justify-content: center;
  flex-wrap: wrap;
  gap: var(--space-md);
}

.hero {
  padding: var(--space-xl) var(--space-md);
}

.skills {
  padding: var(--space-xl) var(--space-md);
}

.skills__grid {
  flex-direction: column; /* columnas apiladas en móvil */
}

/* === Breakpoint 1: Tablet (768px y más) === */
@media (min-width: 768px) {
  .navbar {
    flex-direction: row;
    padding: var(--space-md) var(--space-xl);
    text-align: left;
  }

  .skills__grid {
    flex-direction: row; /* tarjetas en fila en tablet */
  }
}

/* === Breakpoint 2: Desktop (1024px y más) === */
@media (min-width: 1024px) {
  .hero {
    padding: var(--space-2xl) var(--space-md);
  }

  .skills {
    padding: var(--space-2xl) var(--space-xl);
  }
}
```

Verifica los breakpoints en DevTools

Con el emulador de dispositivos abierto, arrastra el borde derecho de la pantalla para cambiar el ancho manualmente. Deberías ver:

- **Menos de 768px** → navbar en columna, tarjetas apiladas
- **768px o más** → navbar en fila, tarjetas en fila
- **1024px o más** → más espaciado en hero y skills

💡 CHECKPOINT – PARTE 3

✅ Antes de continuar, verifica que:

- DevCard se ve correctamente en móvil (360px), tablet (768px) y desktop (1280px)
- Los estilos base están escritos para móvil (sin media query)
- Tienes exactamente dos `@media (min-width: ...)` en tu CSS
- Todos los espaciados del responsive usan variables, no valores hardcodeados

Parte 4: Frameworks de CSS

Los frameworks de CSS son colecciones de estilos prediseñados que aceleran el desarrollo. No reemplazan saber CSS — lo complementan. Vas a reconstruir la tarjeta de habilidades de DevCard dos veces: una con cada framework.

Comparación de Frameworks

Framework	Bootstrap	Tailwind CSS
Estilo	Componentes prediseñados con clases descriptivas	Utilidades atómicas, cada clase hace una sola cosa
Filosofía	Escribe HTML con clases como <code>btn btn-primary</code> y Bootstrap aplica el estilo	Compones el diseño combinando clases como <code>p-4 text-lg font-bold</code>
Ideal para	Prototipos rápidos, interfaces de administración	Diseños personalizados, sin "aspecto Bootstrap"

Parte 4A — Bootstrap

Crea un nuevo archivo `bootstrap-card.html`. Enlaza Bootstrap desde CDN — no necesitas instalar nada.

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>DevCard – Bootstrap</title>
  <!-- Bootstrap desde CDN -->
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
        rel="stylesheet">
</head>
<body class="bg-light">

  <!-- Navbar con Bootstrap -->
  <nav class="navbar navbar-dark bg-primary px-4">
    <span class="navbar-brand fw-bold">DevCard</span>
  </nav>

  <!-- Hero con Bootstrap -->
  <div class="bg-white text-center py-5">
    <h1 class="fw-bold">Ada Lovelace</h1>
    <p class="text-muted">Full-Stack Developer</p>
    <a href="#" class="btn btn-primary mt-2">Ver proyectos</a>
  </div>

  <!-- Tarjetas con Bootstrap Grid -->
  <div class="container py-5">
```

```
<h2 class="mb-4">Mis habilidades</h2>
<!-- row + col-md-4 = 3 columnas en tablet+, 1 en móvil -->
<div class="row g-4">

  <div class="col-12 col-md-4">
    <div class="card h-100 shadow-sm">
      <div class="card-body">
        <div class="fs-1">🌐</div>
        <h5 class="card-title mt-2">HTML & CSS</h5>
        <p class="card-text text-muted">Semántica, Flexbox, Grid y diseño responsivo.</p>
      </div>
    </div>
  </div>

  <div class="col-12 col-md-4">
    <div class="card h-100 shadow-sm">
      <div class="card-body">
        <div class="fs-1">⚙️</div>
        <h5 class="card-title mt-2">JavaScript</h5>
        <p class="card-text text-muted">ES6+, DOM, Fetch API y programación asíncrona.</p>
      </div>
    </div>
  </div>

  <div class="col-12 col-md-4">
    <div class="card h-100 shadow-sm">
      <div class="card-body">
        <div class="fs-1">📦</div>
        <h5 class="card-title mt-2">Node.js</h5>
        <p class="card-text text-muted">APIs REST, Express y manejo de bases de datos.</p>
      </div>
    </div>
  </div>

</div>
</div>

</body>
</html>
```

⚠️ CLASES DE BOOTSTRAP QUE USAMOS

Clase	Equivalente en CSS propio
bg-primary	background-color: #0d6efd
text-muted	color: #6c757d
fw-bold	font-weight: 700
py-5	padding-top: 3rem; padding-bottom: 3rem
col-12 col-md-4	100% en móvil, 33% desde 768px
shadow-sm	box-shadow: 0 1px 3px rgba(0,0,0,.1)

Parte 4B — Tailwind CSS

Crea otro archivo `tailwind-card.html`. Tailwind se puede usar desde CDN con un script — en proyectos reales se instala con npm, pero para explorar esta forma funciona perfecto.

```
<!DOCTYPE html>
<html lang="es">
<head>
```

```
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>DevCard – Tailwind</title>
<!-- Tailwind CDN (solo para explorar, no usar en producción) -->
<script src="https://cdn.tailwindcss.com"></script>
</head>
<!-- En Tailwind, las clases se escriben directamente en el HTML -->
<body class="bg-slate-100">

  <!-- Navbar -->
  <nav class="bg-blue-600 px-8 py-4 flex justify-between items-center">
    <span class="text-white font-extrabold text-xl">DevCard</span>
  </nav>

  <!-- Hero -->
  <div class="bg-white text-center py-16 px-4">
    <h1 class="text-4xl font-extrabold">Ada Lovelace</h1>
    <p class="text-slate-500 mt-2">Full-Stack Developer</p>
    <a href="#"
      class="inline-block mt-6 bg-blue-600 text-white font-bold
        px-8 py-3 rounded-lg hover:bg-blue-700 transition">
      Ver proyectos
    </a>
  </div>

  <!-- Tarjetas -->
  <div class="max-w-4xl mx-auto px-4 py-12">
    <h2 class="text-2xl font-bold mb-6">Mis habilidades</h2>
    <!-- grid-cols-1 en móvil, grid-cols-3 en md+ -->
    <div class="grid grid-cols-1 md:grid-cols-3 gap-6">

      <div class="bg-white rounded-xl p-6 border border-slate-200 shadow-sm">
        <div class="text-4xl">🌐</div>
        <h3 class="font-bold mt-3 mb-1">HTML & CSS</h3>
        <p class="text-slate-500 text-sm">Semántica, Flexbox, Grid y diseño responsivo.</p>
      </div>

      <div class="bg-white rounded-xl p-6 border border-slate-200 shadow-sm">
        <div class="text-4xl">⚙️</div>
        <h3 class="font-bold mt-3 mb-1">JavaScript</h3>
        <p class="text-slate-500 text-sm">ES6+, DOM, Fetch API y programación asíncrona.</p>
      </div>

      <div class="bg-white rounded-xl p-6 border border-slate-200 shadow-sm">
        <div class="text-4xl">📦</div>
        <h3 class="font-bold mt-3 mb-1">Node.js</h3>
        <p class="text-slate-500 text-sm">APIs REST, Express y manejo de bases de datos.</p>
      </div>

    </div>
  </div>
</body>
</html>
```

❗ CÓMO LEER LAS CLASES DE TAILWIND

Clase Tailwind	Equivalente en CSS
bg-blue-600	background-color: #2563eb
text-white	color: #ffffff
font-extrabold	font-weight: 800
px-8 py-4	padding: 1rem 2rem

Clase Tailwind	Equivalente en CSS
<code>rounded-xl</code>	<code>border-radius: 0.75rem</code>
<code>md:grid-cols-3</code>	<code>@media(min-width:768px) { grid-template-columns: repeat(3,1fr) }</code>
<code>hover:bg-blue-700</code>	<code>.btn:hover { background-color: #1d4ed8 }</code>

Comparación Final

Abre los tres archivos (`index.html`, `bootstrap-card.html`, `tailwind-card.html`) en Chrome y observa las diferencias.

Aspecto	CSS propio	Bootstrap	Tailwind
¿Dónde vive el estilo?	Archivo <code>.css</code> separado	Clases en el HTML + CSS de Bootstrap	Solo clases en el HTML
¿Qué tan personalizable es?	Total	Limitado sin sobrescribir	Muy alto con configuración
¿Cuánto CSS hay que saber?	Todo	Poco (las clases lo hacen)	Bastante (las clases son CSS)
¿Responsive incluido?	Manual con <code>@media</code>	Sí, con <code>col-md-</code>	Sí, con prefijos <code>md:</code>
¿Cuándo usarlo?	Siempre como base	Prototipos y dashboards	Proyectos con diseño propio

💡 CHECKPOINT FINAL – PARTE 4

✅ Antes de continuar, verifica que:

- Tienes `bootstrap-card.html` funcionando con las tres tarjetas en columnas responsivas
- Tienes `tailwind-card.html` con el mismo layout usando solo clases de utilidad
- Puedes identificar cuál es más conveniente para distintos tipos de proyecto
- Entiendes que `md:grid-cols-3` en Tailwind es equivalente a `@media (min-width: 768px)`

⚠️ IMPORTANTE SOBRE LOS FRAMEWORKS

Los frameworks de CSS son herramientas, no atajos para no aprender CSS. Si no entiendes qué hace `flex justify-between items-center` en Tailwind o `d-flex justify-content-between` en Bootstrap, no podrás debuggearlos ni personalizarlos. Saber el CSS detrás de las clases siempre es la ventaja.

Recursos Adicionales

Documentación Oficial

- MDN CSS Variables
- Chrome DevTools
- Bootstrap Documentation
- Tailwind CSS Documentation

Ejercicios Extra

- Dark Mode Persistente:** Modifica el JavaScript para guardar la preferencia del usuario en `localStorage`
- Animaciones:** Agrega transiciones suaves al cambiar entre modo claro y oscuro

3. **Grid System:** Reconstruye el layout usando CSS Grid en vez de Flexbox
4. **Framework Comparison:** Crea la misma página con otro framework como Bulma o Foundation

Ejemplos Completos

-  [Ver ejemplo con Bootstrap](#)
-  [Ver ejemplo con Tailwind](#)

Tips para el Proyecto Final

- Usa variables CSS desde el inicio del proyecto
- Define tus breakpoints de responsive consistentemente
- Documenta tu sistema de diseño (colores, espaciados, tipografía)
- Prueba en múltiples dispositivos usando DevTools
- Considera usar un framework si necesitas desarrollo rápido

Conclusión

En esta práctica aprendiste:

1. **DevTools:** Cómo debuggear CSS eficientemente
2. **Variables CSS:** Cómo crear sistemas de diseño mantenibles
3. **Responsive Design:** Cómo hacer que tu sitio funcione en cualquier pantalla
4. **Frameworks:** Cuándo y cómo usar Bootstrap vs Tailwind

Estas habilidades son fundamentales para el desarrollo web moderno. La clave está en entender los conceptos base de CSS antes de saltar a los frameworks.

 **¡Felicidades!** Ahora tienes las herramientas para crear sitios web profesionales y responsivos.